

0.23.2. Sans « x = » : la console range la position — doublé, deux positions

Les deux, sans « = » : la console range x pour le A et xb pour le B.

1. Ce qu'il faut comprendre

C'est le 0.23 et le 0.23.1 réunis : les deux lettres, sans aucun `=`.

La console range chaque position dans SA variable : celle qu'on donne en premier à `deplace_x`, `x` pour le A, `xb` pour le B.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1639 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // le A
```

3. UNE VARIABLE — XB

```
uint8_t xb = 0;     // le B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace_x(x, 0, ALPHABET[0], pas);    // x vaut 5
    deplace_x(xb, 8, ALPHABET[1], pas);   // xb vaut 5
    nombre(0, 2, x);
    nombre(0, 10, xb);

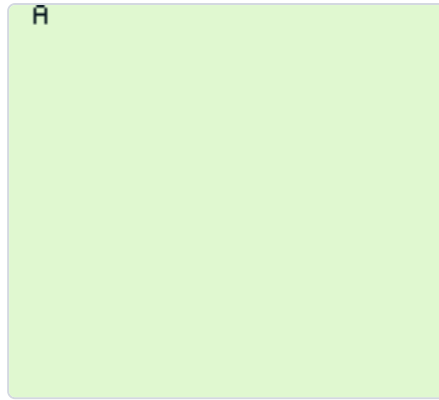
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

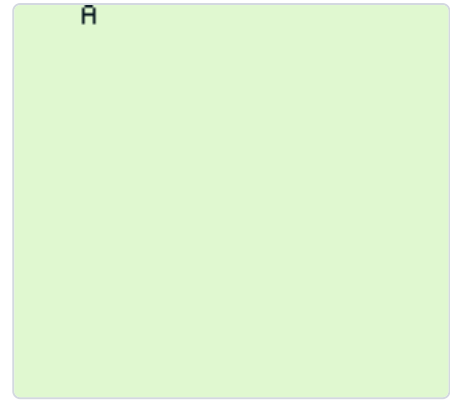
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



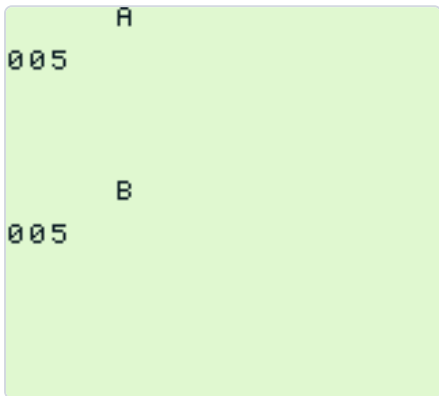
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A puis le B avancent de 5 cases ; sous chacun, 005.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (5, 0) (5,0)

✓ le B finit en (5, 8) (5,8)

✓ 005 et 005

Ensuite : **0.24. Les deux axes à la suite** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++